

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА»

Лабораторная работа №7
Реализация паттернов «Итератор» и «Фабричный метод» для
табулированных функций

Выполнила:

Иванова Елизавета Вадимовна,
студентка группы 6301-030301D

Самара 2025

Задание 1. Реализация паттерна «Итератор»

- Интерфейс `TabulatedFunction` расширен маркером `Iterable<FunctionPoint>`.
- В классах `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` реализован метод `iterator()`, возвращающий анонимный объект типа `Iterator<FunctionPoint>`.
- Итераторы работают **напрямую со внутренней структурой** (массив / связный список), не вызывая публичные методы класса, что обеспечивает высокую производительность.
- Метод `next()` возвращает **копию** объекта `FunctionPoint`, гарантируя инкапсуляцию.
- Метод `remove()` выбрасывает `UnsupportedOperationException`, так как удаление не требуется по условию.

Задание 2. Реализация паттерна «Фабричный метод»

- Создан интерфейс `TabulatedFunctionFactory` с тремя методами `createTabulatedFunction(...)`, соответствующими конструкторам табулированных функций.
- В классах `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` объявлены вложенные публичные статические фабрики:
 - `ArrayTabulatedFunction.ArrayTabulatedFunctionFactory`
 - `LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory`
- В классе `TabulatedFunctions`:
 - Объявлено приватное статическое поле `factory` типа `TabulatedFunctionFactory`.
 - Добавлен метод `setTabulatedFunctionFactory()` для смены стратегии создания.
 - Все методы (`tabulate`, `inputTabulatedFunction` и др.) теперь используют `factory.createTabulatedFunction(...)` вместо прямого вызова `new ....`

Задание 3. Реализация создания объектов через рефлексию

- В классе `TabulatedFunctions` добавлены три перегруженных **рефлексивных метода** `createTabulatedFunction(Class<T>, ...)`.
- Методы используют generic-ограничение `<T extends TabulatedFunction>`, чтобы принимать только допустимые классы.
- Через `Class.getConstructor(...)` находятся конструкторы нужной сигнатуры и создаются экземпляры с помощью `newInstance(...)`.
- Все исключения рефлексии (например, `NoSuchMethodException`) перехватываются и оборачиваются в `IllegalArgumentException`, что упрощает отладку.

```
C:\Users\User\.jdk\openjdk-25.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\Intelli
=== Итератор ===
(0.0; 0.0)
(1.0; 1.0)
(2.0; 4.0)

=== Фабрика ===
LinkedListTabulatedFunction
{(0.0; 1.0), (1.5707963267948966; 6.123233995736766E-17), (3.141592653589793; -1.0)}

=== Рефлексия ===
ArrayTabulatedFunction
{(0.0; 0.0), (5.0; 5.0), (10.0; 10.0)}
LinkedListTabulatedFunction
{(0.0; 0.0), (1.5707963267948966; 1.0), (3.141592653589793; 1.2246467991473532E-16)}

Process finished with exit code 0
```